

INTRODUCTION

I created the Wi-Fi dBm simulator program with the goal of helping people and institutions save time, money, and resources by simulating the prospective coverage from a Wi-Fi router or network of routers on a floor plan of their choosing.

The program had to allow the user to select the type of router they want to use, the ability to place said router or routers, place walls, and lastly generate a realistic heatmap of the prospective coverage that such a floor plan would receive. I ended up fulfilling these requirements in my code and ended up adding some additional features: the ability to simulate a completely custom router with its own custom specifications and error checking for every user input.

As of now the program takes in a user's router specifications, let's the user place walls and routers on the simulation plane, simulates how those routers interact with the walls and other routers, and outputs a heatmap in dBm of the coverage this network of routers (or single router) provides.

OVERVIEW OF INPUTS & OUTPUTS

There are two paths you can go down in this program: choosing to simulate an already existing router or simulating a router with its own custom specifications. You will have different inputs depending on which path you go down.

For the path where you choose an existing router, the program will prompt you to choose between three of the best-selling Wi-Fi routers in 2025. The program shows you the specs of each router and you can type in 1, 2, or 3 to select the respective router. Next, the program will ask if you want to simulate the 2.4 or 5 GHz range. By entering 2.4 or 5 and hitting enter you will select the band you want to simulate. Next, the program will ask you what type of wall you want to use in the simulation. It will list them out and by entering a number between 1 and 3 you will select respective walls associated with the number. Lastly, the program will ask you the level of mess you want to simulate in your room. Once again, you will enter a number between 1 and 3 to select the respective mess level. From there, the inputs will change from inputs in the terminal to inputs in a graphical interface.

By left clicking on the GUI you will place a 1x1 wall; by right clicking on an existing wall, you will delete it, and by middle clicking the GUI you will place a router. Lastly, the program also takes in whether you added or removed a router, which you can do by pressing the "Add router" or "Remove Router" button that are on the bottom of the GUI. From there you press "Enter" and the simulation appears on your screen.

The process for simulating a custom router is very similar. First you enter the frequency band you want to simulate (this time it can be any number), second you enter the transmit band which again can be any number, and lastly you enter the antenna gain which again can be any number. From there, all the steps are the same as when simulating an existing router.

The output the program generates is a heatmap that is overlayed on the user's floor plan. It isn't an image but a graphical user interface window that pops up on the user's screen. The GUI shows the scale the heatmap uses and a blue dot which symbolizes the location of where the Wi-Fi router was placed.

USER-DEFINED FUNCTIONS

There are many functions in this program. Many simply make the graphical user interface work and aren't responsible for any simulation or calculation. Thus, I will focus greatly on the three functions that run the entire simulation. I will still cover the other function after these three.

These three main functions are all in simFuncs.py.

First: `def calc_p0(bandChoice, tx, gain):`

```
def calc_p0(bandChoice, tx, gain):
    freq = bandChoice * 1e9
    p0 = tx + gain - (20 * np.log10(freq) - 147.55)
    return p0
```

Calc_p0 is a function that takes in the variables bandChoice, tx, and gain from the user's input and uses Friis formula at 1 m to calculate p0. The formula is: $P_0 = P_t + G_t - 20 \log_{10}(\text{frequency}) - 147.55$. P_t is transmit power in dBm, G_t is transmitter antenna gain. Frequency is in Hz. These three variables all come from the user's input. The program then just calculates the formula using simple algebraic addition, subtraction, multiplication, and log. Once p0 is calculated, the program returns it.

Next: `def simuSignal(routers, p0, nEmpty, wallGrid, wallType):`

```

def simuSignal(routers, p0, nEmpty, wallGrid, wallType):
    GRID_SIZE = wallGrid.shape[0]
    full_grid = np.full((GRID_SIZE, GRID_SIZE), -9999.0)

    for router in routers:

        temp_grid = np.zeros((GRID_SIZE, GRID_SIZE))

        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):

                d = np.sqrt((r - router[0])**2 + (c - router[1])**2)

                if d == 0:
                    Pr = p0
                else:
                    wall_loss = get_wall_loss_along_ray(router, (r, c), wallGrid, wallType)
                    Pr = p0 - 10 * nEmpty * np.log10(d) - wall_loss

                temp_grid[r, c] = Pr

        full_grid = np.maximum(full_grid, temp_grid)

    return full_grid

```

simuSignal is a function that takes in an array of routers, p0 that was calculated in def calc_p0, wallGrid which is a 2x2 matrix that contains all the walls placed by the user in the simulation plane, and wallType which is the loss caused by 1 wall of the selected type.

First, simuSignal initializes a variable called GRID_SIZE which holds the dimensions of the square simulation plane. It also initializes a grid called full_grid which is outputted at the end of the program. For now, it is filled with a very low number so that later on it will store the maximum signal strength at each coordinate.

Next, the program will do the following for every router in the array of routers. It initializes another array called temp_grid full of zeros which will hold the signal strengths from the respective router.

Now the function will go over every single pixel/index in the 2d array of GRID_SIZE. Here it finds the distance between the router's position and the current coordinate (r, c) using the distance formula. Next, it quickly takes care of the case where d is 0. Assuming d is not zero the function will now call another function called get_wall_loss_along_ray(). I recommend you pause here and read how get_wall_loss_along_ray() works and then come back.

After the function `get_wall_loss_along_ray()` has run and `wall_loss` has been initialized we calculate the signal power at the current coordinate (r, c) using the Log-Distance Path Loss Model formula $P_r = P_0 - 10 \cdot n \cdot \log_{10}(d) - L_{(wall)}$. This is once again just simple algebra.

Lastly, function `simuSignal()` stores the calculated transmit power for the current coordinate in the 2d array `temp_grid`. The program checks whether the newly calculated power at (r, c) is stronger or weaker than what is currently stored and if the newly calculated signal is stronger, it updated the main 2d array `full_grid`.

After running all these steps for every router and every index in the grid, the function returns `full_grid`.

Laslty: `def get_wall_loss_along_ray(router, cell, wallGrid, wallType):`

```
def get_wall_loss_along_ray(router, cell, wallGrid, wallType):

    r0, c0 = router
    r1, c1 = cell

    samples = 300
    visited = set()
    total_loss = 0

    for t in np.linspace(0, 1, samples):
        r = int(r0 + (r1 - r0) * t)
        c = int(c0 + (c1 - c0) * t)

        if r < 0 or c < 0 or r >= wallGrid.shape[0] or c >= wallGrid.shape[1]:
            continue

        if (r, c) not in visited:
            visited.add((r, c))
            if wallGrid[r, c] != 0:
                total_loss += wallType

    return total_loss
```

This function is called inside `simuSignal()` to calculate how many walls are between the current coordinate on the plane and the router. For each wall along the path the function will increase the attenuation along the path.

`r0, c0` are the coordinates of the router and `r1, c1` are the coordinates of the current cell whose attenuation is being calculated. The variable `samples` is the number of steps that the 'ray' will take between the router and the cell. In other words, how many times along the

path from the router to the cell will the program check for a possible wall along that path. Next, the program creates a set() called visited which will make sure that the function doesn't calculate the same wall twice. Total_loss is the variable that holds the sum of all losses along the ray.

Now the program runs a for loop which generates 300 points along the ray for us to check. The program sets r and c to one of those 300 points between the router and the coordinate. The function quickly checks to make sure the current coordinate is within the bounds of the simulation. Lastly, for the individual point (1 of 300) it checks if it is a wall or not, and if it's a wall, increases the total_loss by the wall's attenuation. After going over all 300 points along the ray, the function returns the final total_loss along the ray.

Other functions:

The following functions are all in inAndOut.py:

def user_input():

This function takes in the inputs from the user when the user decides to simulate an existing router. It simply a lot of while loops all asking for the user to input a certain parameter and check to make sure it a valid input. The function returns bandChoice, tx, gain, wallType, and the room's clutter.

def cust_user_input():

cust_user_input() is exactly the same as user_input except it is used to take in inputs from the user the user wants to simulate a custom router.

def plotHeatmap(grid, router, wallGrid=None):

This functions is the one that actually outputs the heatmap on top of the walls that the user drew in the GUI. It uses matplotlib.pyplot to generate a presentation grade chart that includes a legend, title, x-label, and a y-label.

The function sets a range and color palette for the color map to use. It then maps the signal strength values at each pixel as a color. It then overlays the walls inputted by the user and makes them slightly transparent. It also adds a blue dot to symbolize the location of the router. Lastly, it adds a scale to the right of the heatmap.

The following functions are all in gui.py:

Def draw_editor()

This function clears the plot area and draws the wall grid and router markers.

def onclick(event):

The entire purpose of this function is to handle all the mouse clicks that happen on the map. Then, depending on whether the click are left, right, or middle and whether the user is adding a wall, removing a wall, or moving a router it performs the respective action.

def on_key(event):

This function closes the editor when the enter key is clicked.

def add_router_button(event):

The function sets the global flag to True which then changes the next click to be one that places a router rather than placing a wall.

def remove_router_button(event):

Removes the most recently placed router.

def start_editor():

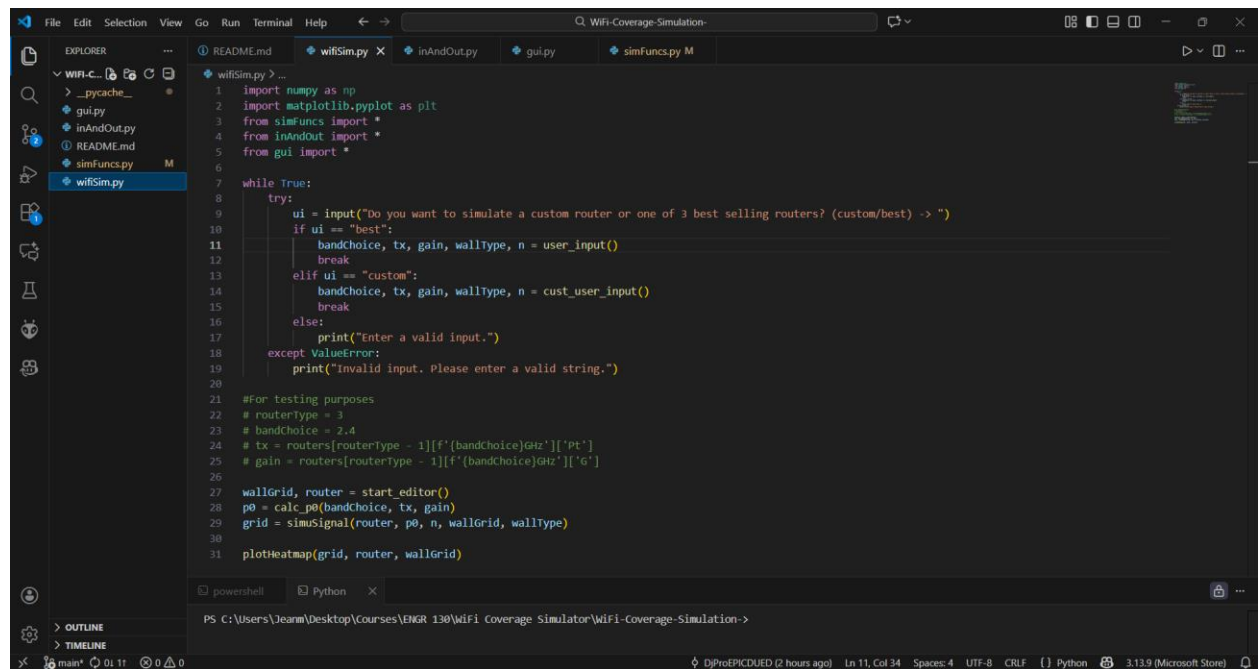
This function runs the GUI by calling matplotlib library.

USER MANUAL

You will need to have Python and the libraries matplotlib and numpy installed for the program to work.

1. Download all the files in the zip file and place them in their own folder.
2. Open that folder with your compiler/IDE. I used VS Code but it should work on any compiler that works with python and has matplotlib and numpy installed.

My environment looks like this:



3. The main .py file that runs and controls the entire program is called wifiSim.py. Make sure it's open. Then run wifiSim.py. From there you can just follow the instructions in the terminal. But don't worry, I'll still walk you through the process here.
4. Once you run wifiSim.py your terminal should look like this:

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> 
```

Use your keyboard to type in "custom" or "best." Then press enter to confirm your choice. From here your path diverges based on whether you chose "custom" or "best."

Path 1: Custom Router Track

1. You chose to simulate a custom router. Now your terminal will ask you the frequency band of your router: For this example, I entered 3.7 but you can enter whatever you want. (I recommend keeping it between 2.4 and 5)

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> custom
Enter you router's frequency band in GHz (usually 2.4 or 5) -> 3.7
```

Press Enter to validate your choice.

2. Now the program asks you to enter a transmit power. Once again enter a number in the recommended range (or any number) and press enter.

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> custom
Enter you router's frequency band in GHz (usually 2.4 or 5) -> 3.7
Enter its transmit power in dBm (usually between 8-30 dBm) -> 32
```

3. Now enter the router's antenna gain. Enter a number into the terminal and press enter.

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> custom
Enter you router's frequency band in GHz (usually 2.4 or 5) -> 3.7
Enter its transmit power in dBm (usually between 8-30 dBm) -> 32
Enter its antenna gain in dBi (usually between 0-10 dBi) -> 3
```

After this step, all future user inputs and console commands will look the same whether you chose custom or best. Scroll down to the **Paths 1 and 2 Merge** section and follow the instructions from there.

Path 2: Existing Router Track

1. You chose to simulate an existing router. Now your terminal will ask you which of the three existing routers you want to simulate. Enter 1, 2, or 3 and press enter. In this example I chose 3.

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> best
Choose one of the following best selling routers to run the simulation on:
1) TP-Link AX1800: 20dBm Tx Power, 4dBi Gain (2.4GHz); 20dBm Tx Power, 5dBi Gain (5GHz)
2) TP-Link Deco X55: 20dBm Tx Power, 3dBi Gain (2.4GHz); 20dBm Tx Power, 4dBi Gain (5GHz)
3) GL-iNet Opal: 19.4dBm Tx Power, 2dBi Gain (2.4GHz); 22.47dBm Tx Power, 2dBi Gain (5GHz)
Enter 1, 2, or 3 -> 3
```

2. Now the program will ask which band you want to simulate. You can only enter 2.4 or 5. I chose 2.4.

```
Do you want to simulate a custom router or one of 3 best selling routers? (custom/best) -> best
Choose one of the following best selling routers to run the simulation on:
1) TP-Link AX1800: 20dBm Tx Power, 4dBi Gain (2.4GHz); 20dBm Tx Power, 5dBi Gain (5GHz)
2) TP-Link Deco X55: 20dBm Tx Power, 3dBi Gain (2.4GHz); 20dBm Tx Power, 4dBi Gain (5GHz)
3) GL-iNet Opal: 19.4dBm Tx Power, 2dBi Gain (2.4GHz); 22.47dBm Tx Power, 2dBi Gain (5GHz)
Enter 1, 2, or 3 -> 3
These routers are dual-band, meaning they operate on both 2.4GHz and 5GHz frequencies.
Would you like to simulate the 2.4GHz or 5GHz band?
Enter 2.4 or 5 -> 2.4
```

3. After this step, all future user inputs and console commands will look the same whether you chose custom or best. Scroll down to the **Paths 1 and 2 Merge** section and follow the instructions from there.

Paths 1 and 2 Merge. This will look the same irrespective of whether you went down path 1 or 2.

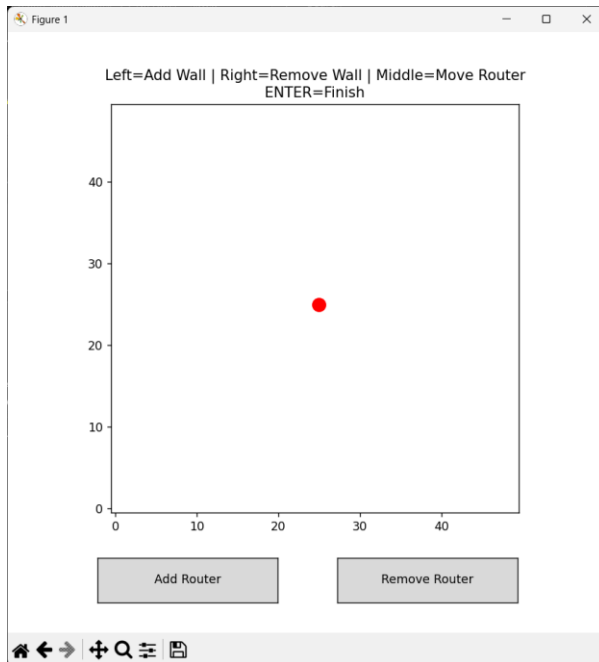
1. Enter a number, either 1, 2, or 3, and press enter to confirm which wall type you want to simulate:

```
Enter the wall type to use for this simulation (1=drywall, 2=brick, 3=concrete) -> 1
```

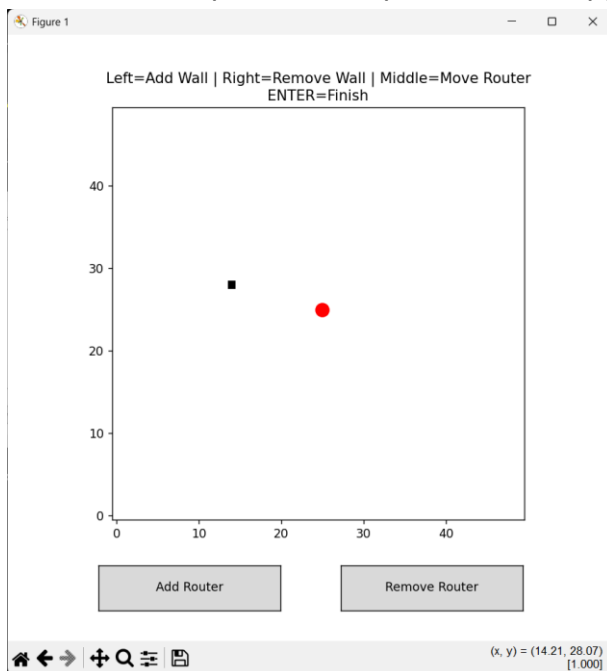
2. Now, once again, enter either 1, 2, or 3, but this time it is to measure the messiness of the room. Press enter to confirm your choice.

```
Enter the wall type to use for this simulation (1=drywall, 2=brick, 3=concrete) -> 1
Do you want to simulate an empty room, a room with medium clutter, or a room with heavy clutter? (1=empty, 2=medium, 3=heavy)-> 1
```

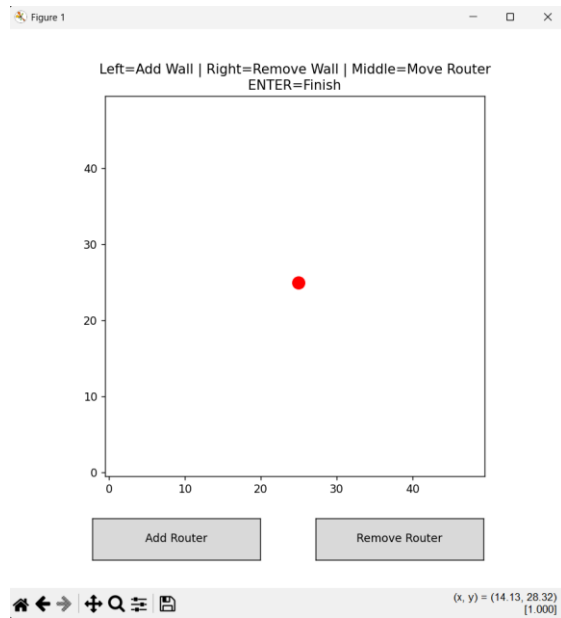

3. After you've pressed enter in step 2, the program will automatically open up a new window. It'll look like this: (The red dot is the Wi-Fi router being simulated)



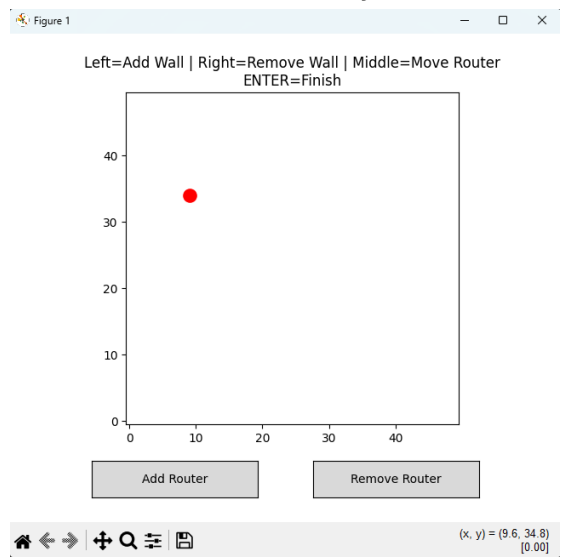
4. Use your left mouse button to add walls to the simulation plane. When you click on the simulation plane, a 1x1 pixel wall will appear where you clicked.



5. You can right click on the wall you just added with your mouse to remove the wall from the simulation plane:

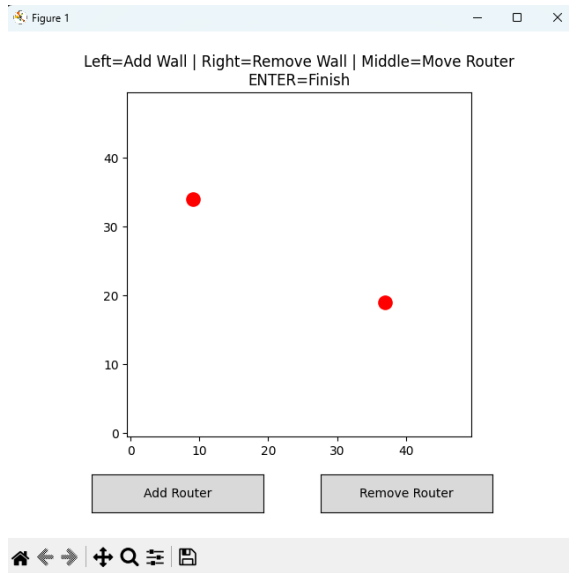


6. You can also use your middle mouse button to move the router around on the simulation plane. Simply click somewhere with the middle mouse button and the router will move to where you clicked.

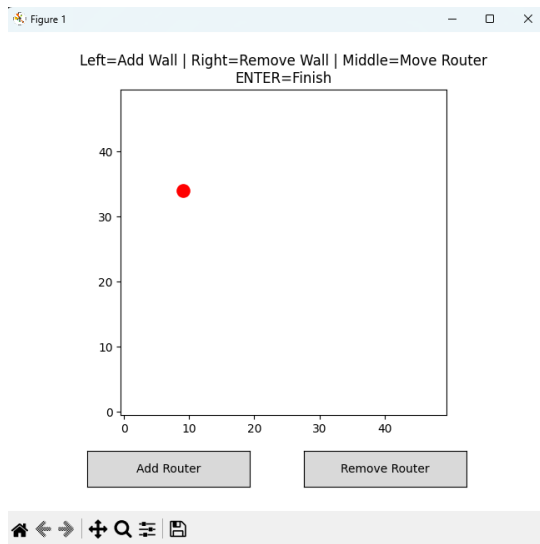


7. To add more routers, you can use the “Add router” button which is right underneath (0,0). After you’ve clicked this button, your next mouse click on the simulation plane will add a new router to the plane. Now when you use the middle mouse button, it

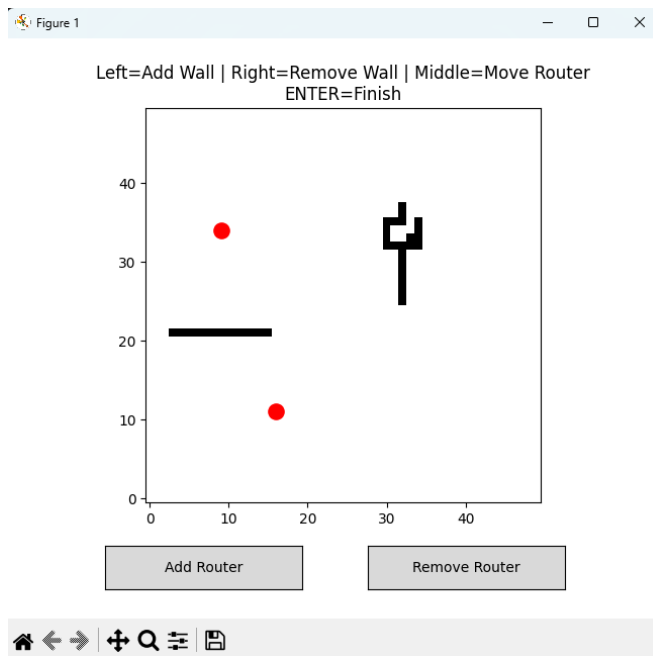
will move the router that is closest to where you clicked.



8. If you now click the “Remove Router” button, it will remove the most recently added router from the simulation plane.



9. Let's add a router and some walls.



10. Now, press the Enter key on your keyboard, the graphical user interface will close, and a new one will appear, this time with a heatmap overlayed on top of the walls and routers that you've placed. You're now done. The program has finished running and you can close the GUI by clicking the 'x' button on the top right corner of the window.

